REPORT TECNICO · PROGETTO FINALE LLM & AGENTIC AI

Casa & Regali — assistente clienti

Versione 2 (v2) · un agente, un passaggio di tool, un secondo passo LLM per la risposta

Abstract. Il progetto realizza un assistente clienti in italiano per un e-commerce fittizio di complementi d'arredo. L'LLM sceglie uno dei sette tool, il tool applica policy e dati reali, un secondo passo LLM scrive la risposta senza poter avviare altre azioni. Il rimborso viene solo *proposto*: entra nel ledger unicamente dopo approvazione umana, con audit JSONL di ogni turno. La v2 riduce l'architettura estesa (supervisore, tre agenti, reflection, Chroma, fallback deterministico) a ciò che con un modello da 1-3B funziona davvero, e misura il risultato su 31 scenari.

31/31	100,0%	45	5,02 s
scenari superati	tool hit rate	test automatici	latenza media

1. Obiettivo e perimetro

La revisione parte da una domanda: **quale parte del sistema ha davvero bisogno dell'LLM?** La risposta è duplice: l'LLM serve per scegliere il tool e per scrivere una risposta in linguaggio naturale. Tutto il resto — idoneità al reso, soglie di rimborso, privacy, approvazione, scrittura del ledger — è codice deterministico e verificabile.

- **Restano** policy-as-code, tool read-only, privacy per cliente, HITL sul rimborso, audit e ledger idempotente.
- **Escono** multi-agente, supervisore, review/reflection, backend deterministico, Chroma e ricerca ibrida.
- **Requisiti coperti**: modello cloud e locale, prompting, tool personalizzati, memoria breve e lunga, RAG, HITL, UI e valutazione.

I requisiti della traccia restano tutti soddisfatti. La differenza è nel modo: invece di aggiungere pattern, il progetto toglie i passaggi che con un modello piccolo producevano latenza e errori, e concentra l'attenzione sul confine tra decisione del modello e vincolo di codice.

2. Architettura

Il grafo ha quattro nodi: **agent** sceglie un tool o risponde; **tools** esegue il tool con il `ToolRuntime` della sessione; **respond** è un secondo passo LLM senza tool che scrive la risposta; **finalize** salva risposta, fonti e tool usati e scrive il turno nell'audit. Se non serve un tool, il percorso è `agent` → `finalize`.

Il ciclo ReAct completo (`agent` → `tools` → `agent` → …) con modelli da 1-3B tende a ripetere lo stesso tool o a concatenarne altri dopo un esito già definitivo. Separare scelta e narrazione riduce latenza e catene incontrollate. Dopo `proponi_rimborso` non possono partire altri tool: l'esito del rimborso è definitivo per il turno.

> **La scelta di progetto**
>
> L'LLM prende due decisioni: quale tool usare e come raccontarne l'esito. Il codice decide tutto il resto. Non c'è una modalità deterministica che sostituisca il modello quando sbaglia: gli errori restano visibili nella valutazione. È una scelta più onesta e più leggibile di un fallback che maschera il comportamento reale.

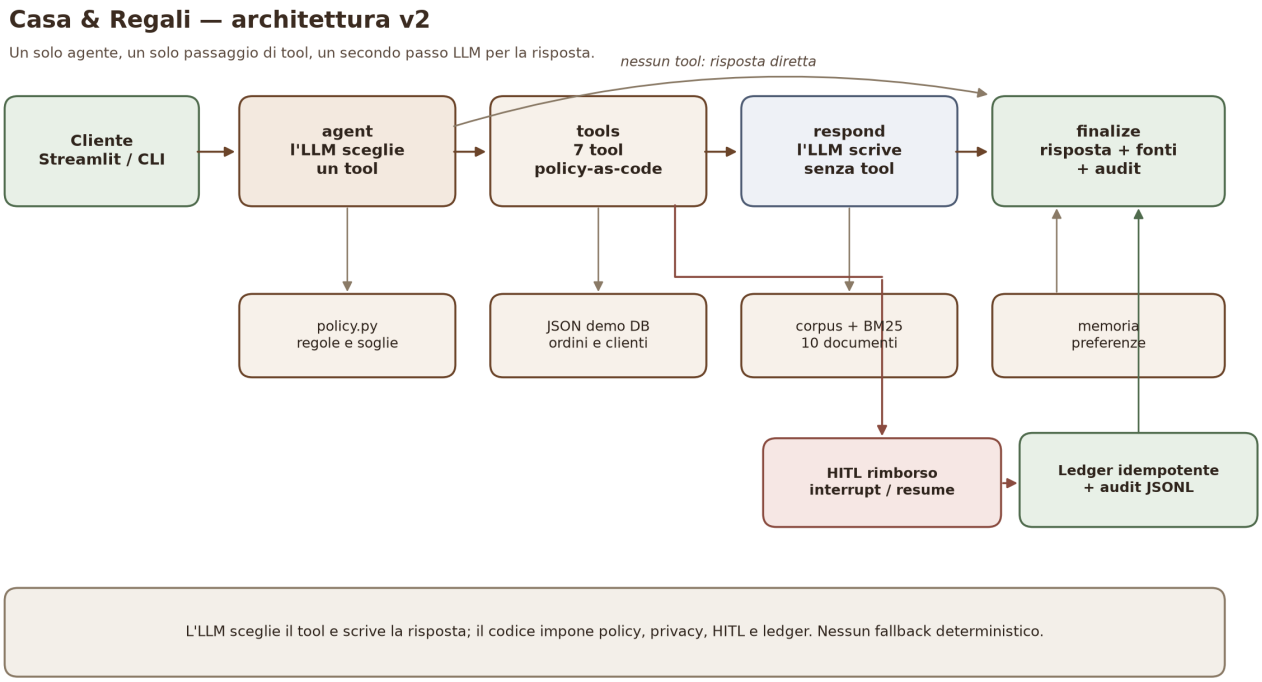


Figura 1 — Flusso agent → tools → respond → finalize. La freccia superiore è il percorso senza tool; il ramo rosso è il rimborso con HITL e ledger.

Cosa fa il modello, cosa fa il codice

Compito	Responsabile
Scegliere il tool e i suoi argomenti	LLM
Scrivere la risposta al cliente	LLM
Applicare finestra resi, esclusioni e soglie	`policy.py`
Garantire il cliente della sessione	`ToolRuntime` + `tools.py`
Sospendere il turno per l'approvazione	`interrupt()`
Scrivere il rimborso solo se approvato	`audit.py` (ledger)
Registrare turni, accessi negati e ticket	`audit.py`

Non esiste una modalità deterministica né un fallback semantico: se il modello sbaglia la scelta, l'errore emerge nella valutazione, non viene mascherato. Questa è la scelta principale della versione: l'AI resta responsabile delle due decisioni che contano, il codice garantisce che non possa violare i vincoli di dominio.

Policy-as-code e privacy

La fonte di verità è `policy.py`: finestra di reso di 30 giorni, categorie escluse (personalizzati, igiene, digitali, fine serie), soglia di escalation di 500 EUR, calcolo del rimborso e controllo duplicati. Il `customer_id` arriva dallo stato del grafo tramite `ToolRuntime`: il modello non può scegliere di quale cliente parlare. Un ordine di un altro cliente restituisce lo stesso messaggio di un ordine inesistente, e l'accesso negato finisce nell'audit.

3. Rimborso, Human-in-the-Loop e ledger

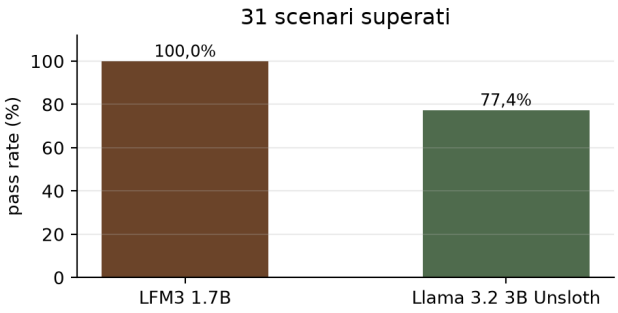
`proponi_rimborso` è l'unico tool che può scrivere un rimborso, e non lo fa direttamente. Verifica l'idoneità; se l'ordine non è idoneo restituisce il motivo e non apre nulla. Altrimenti costruisce la proposta con ordine, cliente, importo, articoli e livello di approvazione, poi chiama `interrupt({"type": "refund_approval", "proposal": …})`. La UI o la CLI mostrano la proposta e riprendono con `Command(resume=…)`. Solo con `approved=True` il ledger rivalida cliente, importo e duplicati, e registra il rimborso.

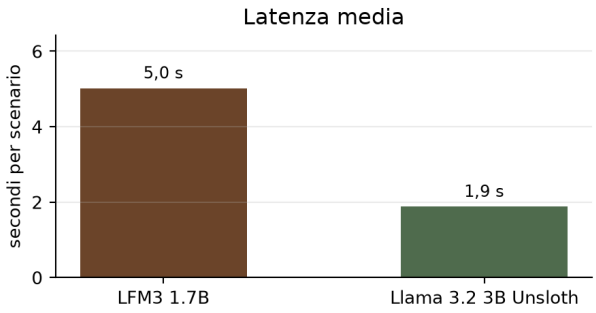
Il ledger non si fida del modello né del chiamante: un importo manomesso, un'approvazione mancante o un rimborso già registrato vengono rifiutati. Sopra i 500 EUR la proposta richiede il livello **responsabile**; la UI mostra una conferma aggiuntiva e la CLI la chiede esplicitamente.

4. Valutazione

La suite contiene **31 scenari**: 9 informazioni, 3 ordini, 6 resi, 6 rimborsi, 3 sicurezza (privacy, ordine inesistente, prompt injection), 2 escalation e 2 memoria. Ogni scenario verifica tool attesi e vietati, contenuto, fonti, interrupt HITL, scritture e assenza di effetti collaterali. I rimborsi usano dati temporanei.

Profilo	Superati	Pass rate	Tool hit	Fonti	HITL	Sicurezza	Latenza
LFM3-1.7B Q4_K_M	31/31	100,0%	100,0%	100,0%	100,0%	100,0%	5,02 s
Llama 3.2 3B Unsloth	24/31	77,4%	96,6%	88,9%	100,0%	66,7%	1,88 s





LFM3 è più lento ma più affidabile: 31/31 con 0,94 tool per turno e 5,02 s medi. Llama (Unsloth) è più veloce (1,88 secondi) ma sbaglia tool, fonti e sicurezza (66,7% contro 100%); i 7 fallimenti si concentrano su resi, memoria e sicurezza.

Reasoning. Con LFM3 il reasoning attivo (variabile vuota) ha dato la migliore scelta dei tool. La risposta finale lo disattiva sempre: con la tabella dei 10 ordini il secondo passo consumava 7.058 token e restituiva contenuto vuoto. Con `LM_STUDIO_REASONING_EFFORT=none` su tutto il grafo una run precedente si è fermata a 27/31. **Gemini.** Nessuna run completa per quota esaurita: il backend resta configurabile, ma senza metrica comparabile.

Perché queste tecniche e non altre

- **Prompting.** Ruolo e vincoli espliciti, few-shot per la scelta del tool e per la risposta diretta, argomenti descritti da schemi Pydantic; il prompt finale riceve solo richiesta ed esiti e vieta calcoli e dati inventati. Poche regole, perché con modelli da 1-3B un prompt lungo diluisce le istruzioni.
- **Un tool per turno, poi `respond` senza tool.** Il ciclo ReAct completo tende a ripetere lo stesso tool o a concatenarne altri dopo un esito definitivo; separare scelta e narrazione riduce latenza e catene incontrollate.
- **BM25, non Chroma.** Il corpus ha dieci documenti brevi e distinti: BM25 non richiede embedding né indice vettoriale, è deterministico e rende le fonti verificabili. Su un corpus più grande servirebbe ricerca semantica o ibrida.
- **Memoria breve e preferenze esplicite.** Il checkpointer tiene il filo della sessione; a lungo termine salviamo solo ciò che il cliente chiede di ricordare, ripulito da email, telefoni e carte. Nessun profilo dedotto senza richiesta.
- **HITL dentro `proponi_rimborso`.** Un nodo separato aggiunge un passaggio e lascia aperta la scrittura da altri percorsi; l'`interrupt` nel tool lega proposta e approvazione, mentre il ledger rivalida cliente, importo e duplicati.
- **Un solo agente, niente multi-agente, reflection o fallback.** Supervisore e review moltiplicano le chiamate senza migliorare la scelta; un fallback nasconderebbe gli errori. Audit JSONL ed eval custom al posto di LangSmith.

5. Test e riproducibilità

La suite automatica usa un modello scriptato (`ScriptedModel`) e dati isolati: non richiede LM Studio né rete. Copre flussi del grafo, HITL, privacy, memoria, policy, corpus, ledger e scenari. Stato attuale: **45 test superati**.

```
python -m pytest -q tests
python eval/run_eval.py --backend local
streamlit run app.py
python cli.py --customer C-001
```

Per riprodurre la valutazione serve LM Studio con il modello caricato: `lms load lfm3-1.7b@q4_k_m --gpu max --context-length 8192 --yes`. I risultati sono in `eval/results/` (JSON e Markdown con failure gallery); `latest.json` contiene l'ultima run. La demo si ripristina con `python scripts/reset_demo.py`.

6. Limiti

- **Modelli piccoli.** LFM3 1,7B e Llama 3.2 3B (Unsloth) possono sbagliare la scelta del tool o riformulare i numeri; la valutazione misura questi errori e non li corregge con fallback.
- **Un tool per turno.** Una richiesta che ne richiede due viene gestita al turno successivo: è una scelta deliberata, non un limite tecnico del grafo.
- **BM25.** Adatto a dieci documenti brevi; su un corpus più grande servirebbe ricerca semantica o ibrida.
- **Memoria di sessione.** `InMemorySaver` perde lo stato al riavvio; le preferenze a lungo termine restano su file.
- **Gemini.** Backend presente ma senza run completa per quota.

7. Confronto con la versione estesa

Aspetto	Versione estesa	v2
Agenti	Supervisore + 3 agenti	1 agente
Ciclo tool	ReAct con più passaggi	Un solo passaggio
Review/reflection	Sì	No
RAG	Chroma + embedding	BM25
Backend deterministico	Sì	No
HITL	Nodo separato del grafo	Dentro proponi_rimborso
Scenari	30	31
Test	76	45

Conclusione

La v2 mantiene i vincoli di dominio e toglie i pattern che, con un modello da 1-3B, non ripagavano la complessità. Il grafo è piccolo abbastanza da essere letto in un minuto, ma il comportamento non è simulato: l'LLM sceglie davvero il tool, scrive davvero la risposta e sbaglia quando il modello sbaglia. La differenza è che gli errori sono visibili nella valutazione, non nascosti da un fallback.

Il 31/31 di LFM3 non è una garanzia di produzione: è la misura di un prototipo su 31 scenari controllati, con un modello locale e dati sintetici. Llama 3.2 3B (Unsloth) mostra che il margine di errore esiste e dipende dal modello. Il secondo passo LLM separato riduce però il perimetro degli errori: la risposta può essere imprecisa, ma non può avviare un rimborso o cambiare cliente.